

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 01 -

SERVIDORES WEB COM NGINX E APACHE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	20
O QUE VOCÊ VIU NESTA AULA?	21
REFERÊNCIAS.....	22

O QUE VEM POR AÍ?

Nesta primeira aula, vamos mergulhar nos fundamentos dos servidores web, entendendo seu papel central na entrega de conteúdo e na arquitetura de aplicações modernas. Você descobrirá como tecnologias amplamente utilizadas no mercado como Apache HTTP Server e Nginx funcionam, quais são suas diferenças arquiteturais e como configurá-los para obter o máximo desempenho em ambientes de produção.

Ao longo da aula, exploraremos conceitos essenciais como forward proxy (proxy direto) e reverse proxy (proxy reverso), entenderemos como otimizar o desempenho com estratégias de cache, configurações SSL/TLS e ainda veremos sobre as boas práticas de segurança. Realizaremos configurações práticas com Docker Compose para simular cenários reais de infraestrutura, preparando você para desafios do dia a dia profissional.

Também abordaremos casos de uso específicos do mercado, como quando grandes empresas escolhem pelo Apache ou pelo Nginx, e como integrar ambos para criar soluções híbridas que aproveitam o melhor de cada tecnologia.

Ao final desta aula, você terá não apenas a compreensão teórica das diferenças e vantagens de cada servidor, mas também a habilidade prática para tomar decisões arquiteturais fundamentadas e implementar soluções robustas de infraestrutura web. Prepare seu ambiente e vamos praticar!

HANDS ON

Nesta aula, você será conduzido(a) por três experimentos práticos que reforçam a teoria apresentada nos vídeos. O primeiro contato ocorre com o Apache, onde configuramos um servidor web simples em Docker, servindo páginas HTML a partir do document root. Essa etapa inicial ajuda a compreender a função de um servidor web e a forma como o Apache organiza seus módulos e diretivas de configuração.

Na sequência, avançaremos para o estudo de Proxy. Vocês experimentarão tanto o Forward Proxy (que atua como intermediário entre o cliente e a internet), quanto o Reverse Proxy (em que o servidor recebe as requisições e decide qual back-end deve processá-las). Essa prática mostra de forma clara como essas duas abordagens diferem e em quais cenários cada uma delas é aplicada na prática

Por fim, exploraremos o Nginx e o Apache lado a lado, comparando arquitetura, desempenho e organização. No Nginx, trabalharemos com server blocks para hospedar múltiplos sites, verificaremos sua eficiência no tratamento de conexões simultâneas e aplicaremos configurações avançadas, como cache de conteúdo e otimizações. Já no Apache, revisaremos a utilização de Virtual Hosts, a reescrita de URLs via .htaccess e o uso de cache. Encerraremos integrando os dois servidores: o Nginx na borda (como proxy reverso com cache), e o Apache no back-end (processando conteúdo dinâmico).

O código-fonte e os exemplos práticos de cada exercício estão organizados no [repositório](#) do curso.

SAIBA MAIS

Servidores Web: Os Fundamentos Além do Básico

O servidor web é a ponte entre o seu navegador e o conteúdo digital. Mas qual é a complexidade por trás dessa "ponte"?

O Protocolo HTTP/HTTPS: A Conversa da Web em Detalhes

O HTTP (Hypertext Transfer Protocol) é a linguagem que navegadores e servidores usam para conversar. No nível fundamental, ele é um protocolo sem estado (stateless), ou seja, cada requisição é tratada de forma independente, sem "memória" de requisições anteriores. Isso o torna escalável, mas exige mecanismos adicionais (como cookies e sessões) para manter o contexto do usuário.

A evolução do HTTP trouxe melhorias significativas:

- **HTTP/1.1:** introduziu conexões persistentes (keep-alive), permitindo que múltiplas requisições e respostas ocorram sobre uma única conexão TCP, reduzindo a sobrecarga de negociação de conexão;
- **HTTP/2:** trouxe multiplexação (múltiplas requisições e respostas simultâneas em uma única conexão), priorização de recursos, compressão de cabeçalhos (HPACK) e Server Push (o servidor envia recursos antes mesmo do cliente pedir). Isso resultou em ganhos notáveis de performance, especialmente em cenários com muitos pequenos recursos;
- **HTTP/3:** a mais recente evolução, baseada no protocolo QUIC (Quick UDP Internet Connections) da Google. QUIC roda sobre UDP em vez de TCP, resolvendo o problema de "head-of-line blocking" em HTTP/2 e melhorando o desempenho em redes com perda de pacotes ou latência alta.

O HTTPS adiciona a camada de segurança TLS/SSL, criptografando a comunicação. A terminação SSL/TLS, onde o processo de criptografia/descriptografia é descarregado para um proxy reverso (como Nginx), é uma prática padrão para otimizar o desempenho dos servidores de back-end, que podem se concentrar no processamento da lógica de negócios.

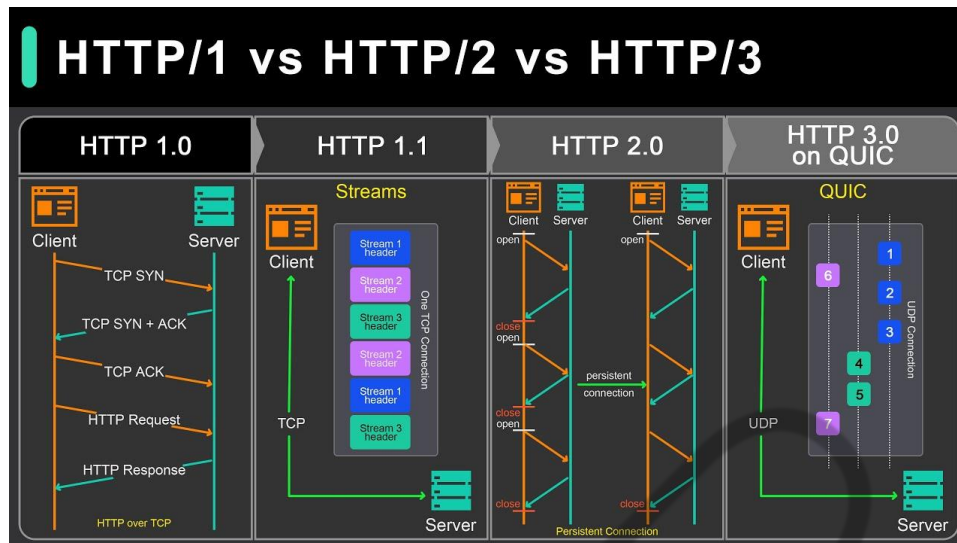


Figura 1 - Comparativo entre HTTP/1.1, HTTP/2 e HTTP/3.
Fonte: <https://i.ytimg.com/vi/UMwQjFzTQXw/maxresdefault.jpg> (s.d)

Evolução e Papel Estratégico do Servidor Web

Historicamente, servidores web eram máquinas simples que apenas entregavam arquivos. Com o advento de aplicações dinâmicas, eles evoluíram para se tornarem orquestradores complexos, capazes de:

- **Servir Conteúdo Estático:** HTML, CSS, JavaScript, imagens, vídeos... esta é a função mais básica, e onde o Nginx se destaca devido à sua arquitetura;
- **Fazer Proxy para Aplicações Dinâmicas:** encaminhar requisições para servidores de aplicação (PHP-FPM, Apache Tomcat, Node.js, Python Gunicorn, etc.) que geram conteúdo dinâmico;
- **Balanceamento de Carga:** distribuir requisições entre múltiplos servidores back-end para garantir alta disponibilidade e escalabilidade;
- **Terminação SSL/TLS:** gerenciar certificados e criptografia;
- **Caching:** armazenar respostas para entregá-las mais rapidamente em requisições futuras;
- **Controle de Acesso e Segurança:** filtrar tráfego malicioso, aplicar regras de autenticação/autorização, proteger contra-ataques comuns (WAF - Web Application Firewall).

A escolha e configuração de um servidor web impactam diretamente a performance, segurança, escalabilidade e custo operacional de uma aplicação.

Proxies em Detalhes: Guardiões e Orquestradores de Tráfego

Vimos que proxies são intermediários. Agora, vamos explorar a fundo o "como" e o "por quê" de suas operações.

Forward Proxy: O Intermediário de Saída com Poderes de Controle

Um Forward Proxy, posicionado entre um cliente (geralmente dentro de uma rede privada) e a internet, atua como um gatekeeper para o tráfego de saída. Ele é explicitamente configurado no cliente (navegador, sistema operacional).

Mecanismos Operacionais:

1. O cliente envia uma requisição ao Forward Proxy, especificando o destino na internet;
2. O proxy intercepta, analisa e, dependendo de suas políticas, decide se encaminha ou bloqueia a requisição;
3. Se encaminhada, o proxy faz a requisição em nome do cliente para o servidor de destino na internet;
4. A resposta do servidor de destino é enviada de volta ao proxy, que a encaminha para o cliente original.

Casos de Uso Avançados:

- **Segurança e Conformidade:** força o tráfego através de firewalls e sistemas de prevenção de intrusão, garantindo que as políticas de segurança da empresa sejam aplicadas. Essencial para ambientes regulados.
- **Controle de Conteúdo:** bloqueio de sites indesejados (redes sociais no horário de trabalho, conteúdo malicioso) e filtragem de downloads.
- **Anonimato e Privacidade:** mascara o endereço IP real do cliente, pois o servidor de destino vê apenas o IP do proxy. Útil para pesquisa de mercado ou proteção da identidade.

- **Cache:** servidores proxy como Squid podem armazenar respostas de requisições frequentes, reduzindo o consumo de banda e acelerando o acesso para múltiplos usuários na mesma rede.

Tipos de Forward Proxy:

- **HTTP Proxy:** mais comum para tráfego web;
- **SOCKS Proxy:** mais flexível, pode lidar com qualquer tipo de tráfego TCP/UDP, não apenas HTTP.

Desafios: pode ser um ponto único de falha e, se mal dimensionado, um gargalo de desempenho.

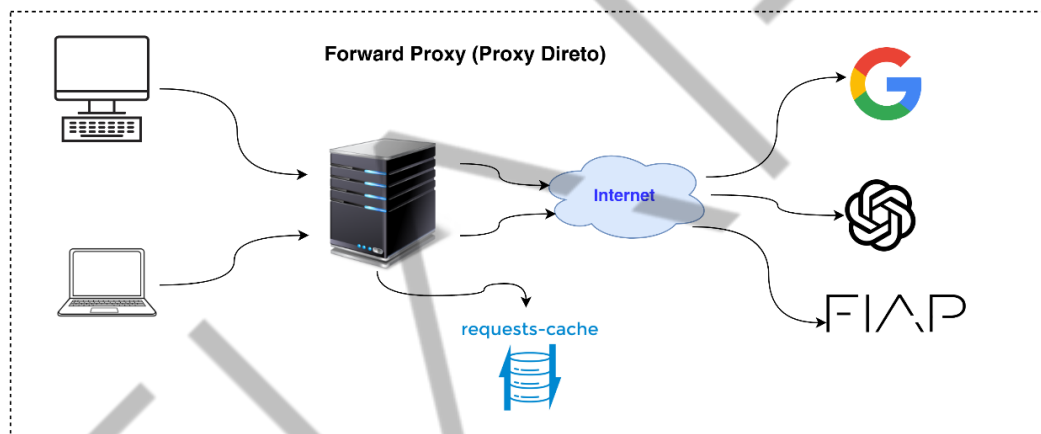


Figura 2 - Exemplo de funcionamento de um Forward Proxy (Proxy Direto)
Fonte: elaborado pelo autor (2025)

Reverse Proxy: O Guardião Inteligente da Entrada

Um Reverse Proxy fica na "borda" da sua infraestrutura, recebendo requisições da internet e distribuindo-as para um ou mais servidores back-end. É transparente para o cliente, que acredita estar se comunicando diretamente com o servidor final.

Mecanismos Operacionais:

1. O cliente envia uma requisição para o Reverse Proxy (IP público);
2. O proxy recebe a requisição e, com base em regras predefinidas (caminho da URL, headers, etc.), escolhe o servidor backend apropriado;
3. O proxy encaminha a requisição ao backend escolhido;

4. O back-end processa a requisição e envia a resposta de volta ao proxy;
5. O proxy recebe a resposta e a envia ao cliente original.

Funções e Casos de Uso Avançados:

Balanceamento de Carga (LoadBalancing): distribui o tráfego entre múltiplos servidores de aplicação idênticos (pool de servidores), garantindo que nenhum servidor seja sobrecarregado e aumentando a disponibilidade. Algoritmos comuns incluem Round Robin, Least Connections, IP Hash.

Terminação SSL/TLS Centralizada: gerencia toda a criptografia para todos os back-ends, simplificando a gestão de certificados e reduzindo a carga de CPU nos servidores de aplicação.

Cache de Conteúdo: armazena respostas para conteúdo frequentemente acessado, aliviando o back-end e acelerando a entrega para o usuário final.

Segurança Aprimorada: atua como a primeira linha de defesa contra ataques DDoS, XSS, SQL Injection, escondendo a topologia da rede interna e filtrando tráfego malicioso. Pode integrar-se com WAFs.

Compactação (Gzip/Brotli): compacta o conteúdo antes de enviá-lo ao cliente, economizando banda e acelerando o carregamento.

Roteamento e API Gateway: em arquiteturas de microsserviços, um reverse proxy pode atuar como um API Gateway, roteando requisições para diferentes serviços com base em critérios complexos.

A/B Testing e CanaryDeployments: direcionar um percentual de usuários para uma nova versão da aplicação, permitindo testes e rollouts controlados.

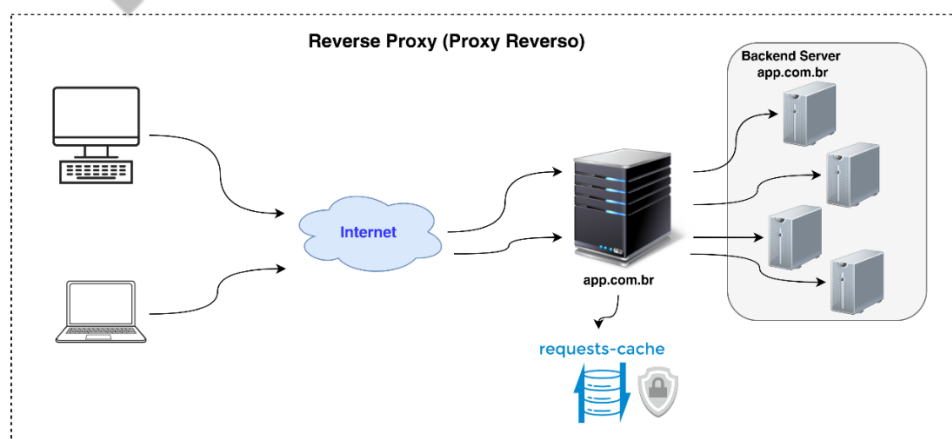


Figura 3 - Exemplo de funcionamento de um Reverse Proxy (Proxy Reverso)
Fonte: Elaborado pelo autor (2025).

Apache HTTP Server: O Canivete Suíço da Web

O Apache, com mais de 25 anos de história, ainda é um gigante resiliente. Sua robustez e flexibilidade vêm de sua arquitetura modular e de processamento.

Modelos de Processamento Múltiplo (MPMs) a Fundo

Os MPMs definem como o Apache lida com as conexões e processos. A escolha do MPM correto é crucial para a performance e o consumo de recursos.

prefork: (Unix, não-threaded)

- **Como funciona:** um processo filho é criado para cada conexão. Cada processo lida com uma única conexão e é isolado.
- **Vantagens:** muito estável, ideal para módulos Apache que não são thread-safe (como mod_php no modo mod_php), alta compatibilidade.
- **Desvantagens:** consome mais memória (cada processo tem sua própria cópia da aplicação), menos escalável para alta concorrência.
- **Uso:** ambientes que precisam de alta estabilidade e compatibilidade com software legado.

worker: (Unix/Windows, multi-threaded)

- **Como funciona:** cria múltiplos processos filhos, e cada processo filho pode ter múltiplos threads. Cada thread lida com uma conexão.
- **Vantagens:** mais eficiente em memória que prefork (threads compartilham memória do processo), melhor performance para alta concorrência.
- **Desvantagens:** problemas em um thread podem afetar todo o processo, módulos devem ser thread-safe.
- **Uso:** aplicações com tráfego moderado a alto, onde a compatibilidade com módulos thread-safe é garantida.

event: (Unix, multi-threaded, otimizado para keep-alive)

- **Como funciona:** baseado no worker, mas com uma otimização chave: threads não ficam presas esperando por requisições keep-alive ociosas. Uma thread pode lidar com várias conexões ao longo do tempo.
- **Vantagens:** mais eficiente para conexões keep-alive e tráfego de alta concorrência, otimizado para HTTP/2.
- **Desvantagens:** exige módulos thread-safe e pode ser mais complexo de configurar.
- **Uso:** servidores com alto volume de conexões persistentes, especialmente com HTTP/2. É o MPM recomendado para a maioria dos novos deployments.

MPM	Uso de Memória	Uso de Memória	Vantagens	Desvantagens	Melhor para / Compatibilidade
Prefork	Processos independentes , não-threaded.	Alto (cada processo é isolado).	Estabilidade, compatibilidade e com módulos não thread-safe (ex: mod_php).	Pouco escalável, alto consumo de memória.	Aplicações legadas ou que exigem máxima compatibilidade.
Worker	Processos com múltiplos threads.	Moderado .	Mais eficiente em memória, melhor desempenho para alta concorrência.	Requer módulos thread-safe, problemas em threads afetam o processo.	Aplicações modernas com tráfego moderado a alto.
Event	Threads com otimização de keep-alive.	Baixo (altamente eficiente).	Ótimo para conexões persistentes, escalabilidade, suporte HTTP/2.	Mais complexo, exige módulos thread-safe.	Ambientes de alta concorrência e aplicações escaláveis.

Tabela 1 - Tabela comparativa dos MPMs do Apache (Prefork, Worker e Event)

Fonte: Elaborado pelo autor (2025)

mod_rewrite: A Arte da Reescrita de URLs

O mod_rewrite é um módulo extremamente poderoso que permite manipular URLs em tempo real com base em expressões regulares. É a espinha dorsal de URLs "amigáveis" e de muitas regras de segurança.

Diretivas Chave:

RewriteEngine On: ativa o motor de reescrita.

RewriteRule: define a regra de reescrita. Ex: RewriteRule ^produto/([0-9]+)\$ /produto.php?id=\$1 [L,NC].

- ^produto/([0-9]+)\$: Padrão Regex para capturar URLs como /produto/123.;
- /produto.php?id=\$1: URL de substituição, onde \$1 é o valor capturado pelo regex;
- Flags: [L] (*Last Rule - para de processar outras regras*), [R=301] (*Redirecionamento Permanente*), [NC] (*No Case - não diferencia maiúsculas/minúsculas*), [QSA] (*Query String Append - anexa parâmetros da URL original*).

RewriteCond: define uma condição para que a RewriteRule seja aplicada. Ex: RewriteCond %{HTTPS} off (se a conexão não for HTTPS).

.htaccess - Flexibilidade e Desafios: o .htaccess permite que desenvolvedores definam regras de mod_rewrite (e outras configurações) em nível de diretório, sem acesso à configuração global do servidor.

- **Prós:** flexibilidade para desenvolvedores, útil em hospedagens compartilhadas.
- **Contras:** cada requisição em um diretório com .htaccess força o Apache a procurar e processar o arquivo, o que gera overhead e pode impactar significativamente a performance em alta escala. É recomendado mover regras .htaccess para os Virtual Hosts sempre que possível em ambientes de produção.

mod_cache: Cache Nativos do Apache

O Apache também possui capacidades de cache (mod_cache). Pode armazenar conteúdo em disco (mod_cache_disk) ou em memória (mod_cache_socache).

- **Configuração:** requer CacheRoot (diretório para cache em disco), CacheEnable (ativa cache para URLs), e diretivas para tempo de expiração (CacheMaxExpire, CacheMaxFileSize).
- **Considerações:** embora funcional, em arquiteturas modernas, o cache é frequentemente realizado por um reverse proxy (como Nginx) na "borda" da rede para maior eficiência.

Nginx: O Velocista e Orquestrador Eficiente

O Nginx foi projetado para alta performance e concorrência, especialmente para servir conteúdo estático e atuar como proxy.

Arquitetura Event-Driven: A Chave da Eficiência

A principal diferença do Nginx é sua arquitetura assíncrona, orientada a eventos. Diferente dos modelos de threads/processos dedicados por conexão:

- **Não-bloqueante (Non-blocking I/O):** um processo Nginx (worker) não espera que uma conexão termine. Ele registra um evento e se move para a próxima conexão. Quando o evento está pronto (dados recebidos, arquivo lido), ele é notificado e retoma o processamento.
- **Event Loop:** cada processo worker tem um "event loop" que gerencia milhares de conexões simultaneamente com poucos recursos. Isso o torna extremamente eficiente para conexões keep-alive e para cenários com muitas conexões "ociosas".

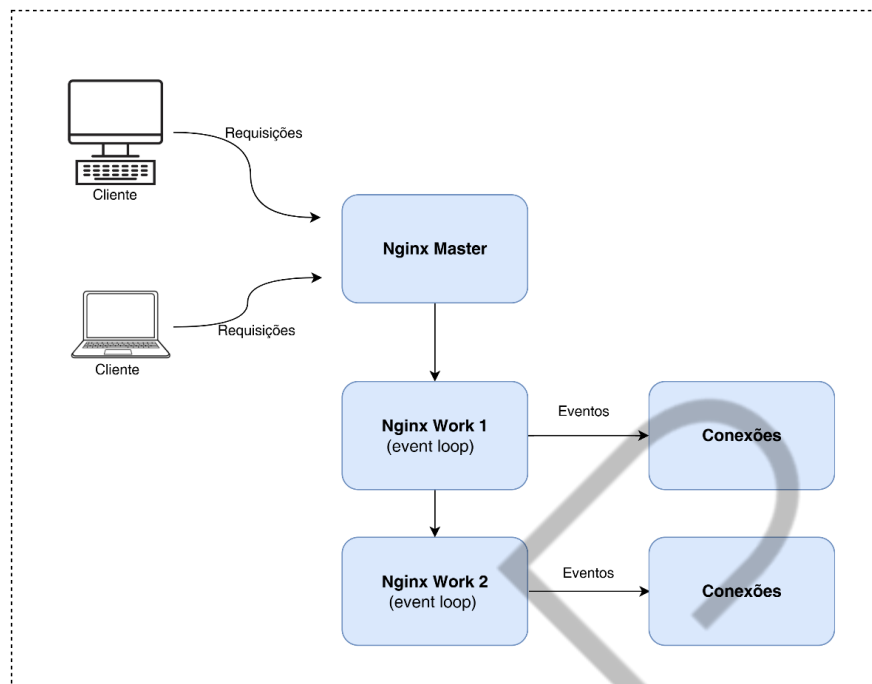


Figura 4 - Arquitetura event-driven simplificada do Nginx
Fonte: Elaborado pelo autor (2025)

locationBlocks Detalhados: O Roteador Granular

No Nginx, o roteamento é definido por server blocks (equivalentes a Virtual Hosts) e, dentro deles, por locationblocks. A ordem e o tipo de correspondência dos locationblocks são cruciais:

Tipos de Correspondência (e ordem de prioridade):

1. = (correspondência exata): Mais alta prioridade. Ex: location = /login { ... }.
2. ^~ (prefixo, preferencial): Se a correspondência prefixo for a mais longa, e usada. Ex: location ^~ /images/ { ... }.
3. ~ ou ~* (expressão regular): Avaliadas na ordem em que aparecem no arquivo. ~ é case-sensitive, ~* é case-insensitive. Ex: location ~ /\.php\$ { ... }.
4. Nenhum modificador (prefixo): Mais baixa prioridade. Ex: location / { ... }.
5. / (catch-all): A mais baixa prioridade, lida com tudo que não foi correspondido.

Exemplo:

```
http {
    server {
        listen 80;
        server_name exemplo.local;
        root /usr/share/nginx/html;
        index index.html index.htm;

        # 1) EXATO (=)
        # Somente /login cai aqui (nem /login/ nem
/login?x=1)
        location = /login {
            return 301 https://$host/login;
        }

        # 2) PREFIXO PREFERENCIAL (^~)
        # Qualquer caminho que comece com /images/ usa
este bloco
        # e NÃO tenta regex depois (por isso é
"preferencial").
        location ^~ /images/ {
            # Ex.: conteúdo estático de imagens
            expires 7d;
            add_header Cache-Control "public, max-age=604800";
            try_files $uri =404;
        }
    }
}
```

```
# 3) REGEX (case-insensitive) ~*
# Bloqueia acesso direto a arquivos sensíveis (.ini,
.env, .sql)
location ~* \.(?:ini|env|sql|bak)$ {
    deny all;
}

# 4) PREFIXO "normal" (sem modificador)
# Rota administrativa prioritária sobre o
catch-all
location /admin/ {
    # Ex.: exige header de autenticação vindo
de um WAF/reverso
    if ($http_x-auth != "ok") { return 403; }
    try_files $uri $uri/ /admin/index.html;
}

# 4) PREFIXO "normal" (API)
# Encaminha /api/ para um backend (reverse
proxy)
location /api/ {
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header                                X-Forwarded-For
$proxy_add_x_forwarded_for;
    proxy_pass http://api-backend:8080/;
}

# 5) CATCH-ALL
# Tudo o que não casou acima cai aqui
location / {
    # try_files: serve arquivo se existir; senão, manda
para index.html
    try_files $uri $uri/ /index.html;
}
}
```

proxy_cache: O Cache Robusto do Nginx

A capacidade de cache do Nginx (proxy_cache) é uma de suas maiores vantagens como reverse proxy, reduzindo a carga nos back-ends e acelerando a entrega.

Diretivas Chave:

- **proxy_cache_path:** define o diretório de cache em disco, níveis de diretórios, tamanho da zona de memória (keys_zone), tempo de

inatividade (inactive), tamanho máximo do cache (max_size) e se usa caminho temporário (use_temp_path=off).

- **proxy_cache:** ativa o cache para um location block, referenciando a zona definida.
- **proxy_cache_valid:** define por quanto tempo diferentes códigos de status HTTP (200, 404, etc.) devem ser cacheados.
- **proxy_cache_bypass:** não armazena nem serve do cache se certas condições forem atendidas (ex: se um cookie de sessão existe).
- **add_header X-Proxy-Cache \$upstream_cache_status;** adiciona um cabeçalho de resposta para depuração, mostrando se a resposta veio do cache (HIT), não veio (MISS), etc.

Exemplo:

```
http {
    proxy_cache_path /var/cache/nginx levels=1:2
    keys_zone=meu_cache:10m
    max_size=100m
        inactive=60m
    use_temp_path=off;

    server {
        listen 80;
        server_name localhost;

        location /api/ {
            proxy_pass http://backend:8080;

            # Ativa cache
            proxy_cache meu_cache;

            # Define tempo de cache para respostas
            proxy_cache_valid 200 302 10m;
            proxy_cache_valid 404 1m;

            # Cabeçalho de depuração
            add_header X-Cache-Status $upstream_cache_status;
        }
    }
}
```

or

- **Nginx na Borda (Frontend):** atua como reverse proxy, balanceador de carga, servidor de conteúdo estático, terminador SSL/TLS. Lida com a alta concorrência e o tráfego de entrada de forma eficiente.
- **Apache no Backend (Back-end):** Servidor de aplicações dinâmicas. Recebe requisições do Nginx e foca no processamento de scripts (PHP, Python, Ruby, Java). Continua usando .htaccess para certas aplicações.

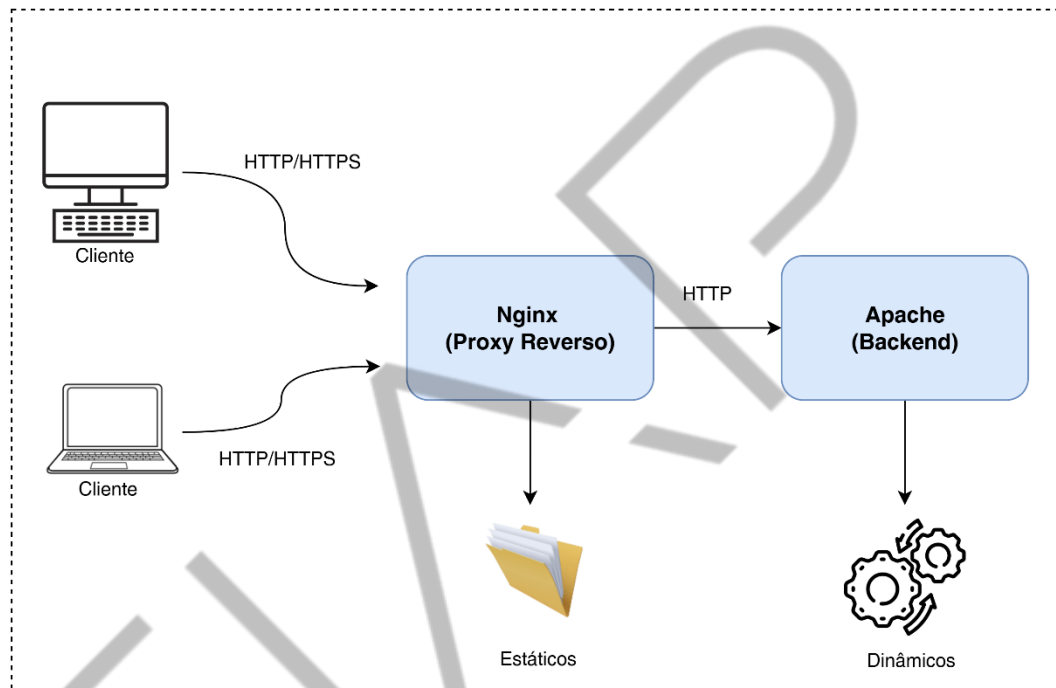


Figura 5 - Arquitetura Híbrida com Nginx e Apache
Fonte: Elaborado pelo autor (2025)

Comunicação e Otimizações em Ambientes Híbridos

- **proxy_pass no Nginx:** a diretiva principal para encaminhar tráfego do Nginx para o Apache.
- **Headers X-Forwarded-*:** cruciais para que o Apache no back-end "saiba" o IP real do cliente, o protocolo original (HTTP/HTTPS) e outras informações importantes, já que a requisição está vindo do Nginx e não diretamente do cliente.
 - X-Real-IP: IP do cliente original.
 - X-Forwarded-For: Lista de IPs de proxies pelos quais a requisição passou.

- X-Forwarded-Proto: Protocolo original (http ou https).
- **Otimização de Conexões:** configurar `keepalive_timeout` e `proxy_read_timeout` no Nginx e `Timeout` no Apache para gerenciar o ciclo de vida das conexões de forma eficiente;
- **FastCGI (PHP-FPM):** para aplicações PHP, a comunicação entre Nginx e PHP-FPM (via FastCGI) é mais performática que passar por um Apache com `mod_php`. Nginx pode se comunicar diretamente com PHP-FPM, e o Apache pode ser eliminado do stack para PHP puro, ou mantido para outras tecnologias.

Considerações Avançadas e Próximos Passos

Para quem deseja se aprofundar ainda mais, considere explorar:

Monitoramento: ferramentas como Prometheus e Grafana para coletar métricas e visualizar o desempenho de seus servidores em tempo real.

Automação e IaC (Infrastructure as Code): ferramentas como Ansible, Terraform, Puppet ou Chef para automatizar a configuração e o deployment de servidores.

Containerização e Orquestração: docker e Kubernetes para empacotar e gerenciar suas aplicações e servidores de forma escalável.

Segurança: implementação de Web Application Firewalls (WAFs) mais avançados (ex: ModSecurity), integração com sistemas de detecção de intrusão (IDS/IPS), e práticas de hardening de servidores.

A jornada no universo dos servidores web é contínua e fascinante. Com os fundamentos e o aprofundamento que você adquiriu, estará preparado(a) para os desafios e inovações do mundo de DevOps.

MERCADO, CASES E TENDÊNCIAS

Nginx Essentials – Valery Kholodkov, PacktPublishing, 2018

Este livro oferece uma introdução prática e objetiva ao Nginx, abordando desde a configuração básica até funcionalidades avançadas como proxy reverso, cache e balanceamento de carga. Ideal para profissionais que desejam implementar soluções de alta performance com Nginx em produção.

Pro Apache – Peter Wainwright. Apress, 2007.

Considerado uma referência completa sobre o Apache HTTP Server, cobre desde conceitos fundamentais até configurações avançadas de módulos e integração com linguagens dinâmicas. É indicado para administradores e desenvolvedores que trabalham com aplicações complexas no backend.

NGINX Cookbook – Igor Sysoev (prefácio), NGINX Team.F5 Networks, 2020.

Reúne uma coleção de “receitas” práticas para resolver problemas do dia a dia com Nginx, incluindo segurança, caching, balanceamento de carga e integração com microsserviços. Foi elaborado pelo próprio time da NGINX, garantindo relevância e aplicabilidade no mercado.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você compreendeu o papel fundamental dos servidores web e a distinção crucial entre Forward e Reverse Proxies. Aprofundou-se nas arquiteturas e funcionalidades do Apache e Nginx, comparando suas forças e cenários de uso. Adquiriu experiência prática na configuração, otimização e integração desses servidores, especialmente com Nginx atuando como proxy reverso para Apache, preparando-o para desafios reais do mercado.

EMAN

REFERÊNCIAS

APACHE. **Documentação oficial do Apache HTTP Server**. [s.d.]. Disponível em: <https://httpd.apache.org/docs/> Acesso em: 01set. 2025.

NGINX. **Blog oficial da empresa NGINX**. [s.d.] Disponível em: <https://www.nginx.com/blog/> (Acesso em: 01set. 2025).

NGINX. **Documentação oficial do Nginx**. [s.d.]. Disponível em: <https://nginx.org/en/docs/> Acesso em: 01 asetgo. 2025.

EMANIP

PALAVRAS-CHAVE

Servidores Web; Nginx; Apache HTTP Server; Proxy Reverso; Balanceamento de Carga; Alta Disponibilidade.

EMSE



POS TECH